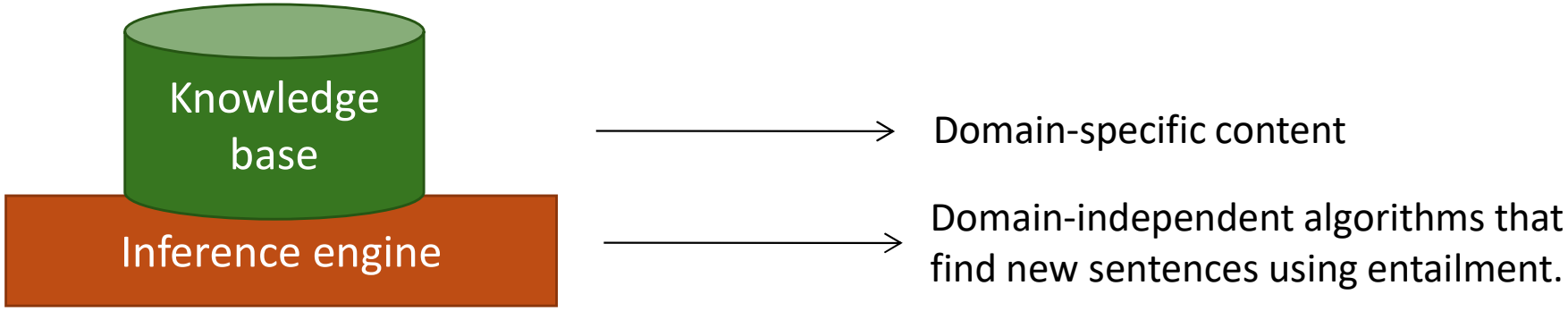
Module 4

Knowledge-Based Agents

Let us introduce a different class of agents that relies on reasoning using knowledge.

Knowledge-Based Agents



- Knowledge base (KB) = **set of facts.** E.g., set of **sentences** in a **formal language** that are known to be true related to the context of the problem.
- **Separation** between data (knowledge) and program (inference).
- **Inference** = deriving new information from information that we already know.
- The concepts of **goals or utility** can be incorporated into the knowledge base as facts.
- **Actions** are based on knowledge (sentences + inferred sentences).

What is a Knowledge Base?

- A **Knowledge Base**, or **KB**, is a collection of information that the agent has stored about the world.
- For example:
  - Fact 1: Ravi is a student.
  - Fact 2: All students study.
  - Fact 3: Ravi studies AI.
- These statements together form part of a knowledge base. We could write:
- KB = {

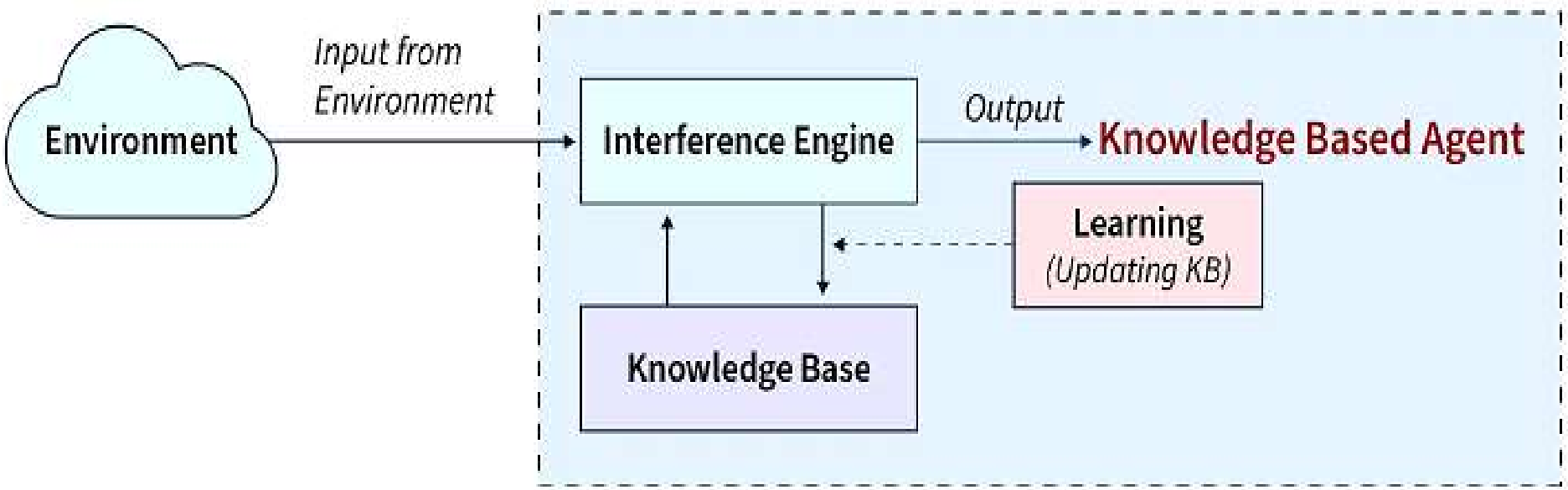
  Ravi is a student,

  All students study,

  Ravi studies AI

}
- The KB can contain: **facts** , **rules** , **relationships** , **information about objects, information about situations**

Generic Knowledge-based Agent



function KB-AGENT(*percept*) **returns** an *action*
  persistent: KB, a knowledge base
    t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(*percept*, t))
  action $\leftarrow$ ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(*action*, t))
  $t \leftarrow t + 1$
  return *action*

Memorize percept at time t

Ask for logical action given the knowledge

Record action taken at time t

Knowledge-based Agent

- A **Knowledge-Based Agent (KBA)** is an agent that uses a **Knowledge Base (KB)** to store information about the world and uses **reasoning/inference** to decide what to do.

Core Components

- **Knowledge Base (KB):** The central repository of domain facts, rules, and long-term beliefs expressed in a formal language.
- **Inference Engine:** The domain-independent reasoning mechanism that applies logical rules to the KB to deduce new facts.
- **Learning Element:** Updates and expands the knowledge base over time based on new experiences.

The Operational Cycle

- **TELL:** The agent feeds new information or percepts from the environment into the knowledge base.
- **ASK:** The agent queries the knowledge base to determine what action it needs to take next.
- **PERFORM:** The agent executes the chosen action and feeds the results back into the system.

Different Languages to Represent Knowledge

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

Logical Agents

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

- Facts are logical sentences that are known to be true.
- Inference: Generate new sentences that are entailed by all known sentences.

Propositional Logic

Logic to Represent Knowledge



Logic is a formal language for representing knowledge and a formal system for reasoning about that knowledge.



Syntax: rules for constructing valid sentences

E.g., $x + 2 \geq y$ is a valid arithmetic sentence, $\geq x2y +$ is not



Semantics: "meaning" of sentences, or relationship between logical sentences and the real world

Specifically, semantics defines truth of sentences

E.g., $x + 2 \geq y$ is true in a world where x 5 and y = 7

A **proposition** is a statement that can be either: **True or False**

Propositional Logic: Syntax in Backus-Naur Form

$$
\begin{aligned}
Sentence &\rightarrow AtomicSentence \mid ComplexSentence \\
AtomicSentence &\rightarrow True \mid False \mid P \mid Q \mid R \mid \ldots \\
ComplexSentence &\rightarrow (\ Sentence\ ) \\
&\mid \neg\ Sentence \\
&\mid Sentence \wedge Sentence \\
&\mid Sentence \vee Sentence \\
&\mid Sentence \Rightarrow Sentence \\
&\mid Sentence \Leftrightarrow Sentence \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

= Symbols

Negation
Conjunction
Disjunction
Implication
Biconditional

Simple English Examples to Propositional Logic

- **Statements**
- "It is raining.“ : → **P**
- "It is not raining.“: → **¬P**
- "It is raining and the ground is wet.“: → **P ∧ Q**
- "It is raining or the ground is wet.“: → **P ∨ Q**

Let:

- P = "It is raining."
- Q = "I carry an umbrella."
- R = "I get wet."
- **Statement 1 :** If it is raining, I carry an umbrella: **P → Q**
- **Statement 2:** If it is raining and I do not carry an umbrella, I get wet: **(P ∧ ¬Q) → R**

Simple English Examples to Propositional Logic

English Statement: If it is raining and the ground is wet, then I carry an umbrella.

Propositional Logic Representation:

- P = "It is raining."
- Q = "The ground is wet."
- R = "I carry an umbrella."

Then:

- **$(P \land Q) \rightarrow R$**

Simple English Examples to Propositional Logic

Let:

- P = "The student studies."
- Q = "The student attends class."
- R = "The student passes."

- If the student studies and attends class, then the student passes: **$(P \land Q) \rightarrow R$**
- The student passes if and only if the student studies and attends class: **$R \leftrightarrow (P \land Q)$**

What is Semantics?

Semantics deals with the **meaning** of sentences.

- Syntax tells us: Is the sentence correctly formed?
- Semantics tells us: What does the sentence mean, and is it true or false?
- Example: **P ∧ Q** is syntax.
- The truth value of **P ∧ Q** under a particular interpretation is semantics.
- An **interpretation** assigns a truth value to each proposition symbol.
- Suppose:
- P = "It is raining.“ Q = "The ground is wet."
- One interpretation could be:
- **P = True Q = True**
- Another: **P = True Q = False**
- Each assignment represents a possible situation.

Possible Worlds, Models and Truth Tables

A **model** specifies a "possible world" with the true/false status of each proposition symbol in the knowledge base

- E.g., **P** is true and **Q** is true
- With two symbols, there are $2^2 = 4$ possible worlds/models, and they can be enumerated exhaustively using:

A **truth table** specifies the truth value of a composite sentence for each possible assignments of truth values to its atoms. Each row is a model.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
$false$	$false$	$true$	$false$	$false$	$true$	$true$
$false$	$true$	$true$	$false$	$true$	$true$	$false$
$true$	$false$	$false$	$false$	$true$	$false$	$false$
$true$	$true$	$false$	$true$	$true$	$true$	$true$

We have 3 possible worlds for $P \Rightarrow Q = true$

Logical Equivalence Rules

Two sentences are logically equivalent iff (read if, and only if) they are true in same models

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Satisfiability, Tautology, Contradiction

- A sentence is **satisfiable** if there exists at least one model in which the sentence is true.
- Example: **P** ∨ **Q** is satisfiable.
- Why? Because at least one model makes it true.
- Example: **P** ∧ **¬P** is **not satisfiable**. There is no possible assignment where both P and ¬P are true.

Tautology

- A sentence that is true in **every model**.
- Example: **P** ∨ **¬P**

Contradiction

- A sentence that is false in **every model**.
- Example: **P** ∧ **¬P**

Validity and Satisfiability

A sentence is **valid** if it is true in **all** models/worlds

e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$ are called tautologies and are useful to deduct new sentences.

A sentence is **satisfiable** if it is true in **some** model

e.g., $A \vee B$, C
useful to find new facts that satisfy all current possible worlds.

A sentence is **unsatisfiable** if it is true in no models

e.g., $A \wedge \neg A$

Entailment

Entailment is one of the most important concepts in logic.

- Notation: **KB ⊨ α**
- Read as: "Knowledge Base KB entails sentence α."
- It means: **α is true in every model in which KB is true.**

Example of Entailment: Knowledge Base:
1. P → Q 2. **P**
Query: **Q**
We ask: Does KB entail Q?
Consider all models where KB is true. The second KB sentence P eliminates the models where P is false. Therefore, every model of KB has: **Q = True**
Hence: **KB ⊨ Q**

P	Q	P→Q	KB true?
T	**T**	**T**	**Yes**
T	F	F	No
F	T	T	No
F	F	T	No

Entailment

- **Entailment** means that a sentence follows from the premises contained in the knowledge base:

$$\boldsymbol{KB} \models \boldsymbol{\alpha}$$

- The knowledge base *KB* entails sentence α iff α is true in all models where *KB* is true

Inference

- **Logical inference:** a procedure/algorithm for generating sentences that follow from (or entailed by) a knowledge base KB.

- An inference procedure is **sound** if it derives a sentence α iff KB ⊨ α. I.e, it only derives **true sentences**.

- An inference procedure is **complete** if it can derive **all** α for which *KB* ⊨ α.

Entailment vs Inference in Propositional Logic

- **Entailment** asks: **"Does the knowledge base logically guarantee that this statement is true?"**
- It is represented as: $KB \models \alpha$ Read as: **KB entails α**
- This means **α is true in every model in which KB is true**.

- **Inference** asks: **"How can we derive a conclusion from the knowledge we have?"** It is the **reasoning process or algorithm** used to obtain a conclusion.
- An inference algorithm is **sound** if it never derives an incorrect conclusion.
- If the algorithm derives α, then: $KB \vdash \alpha \Rightarrow KB \models \alpha$. In simple words: **Whatever the algorithm proves must actually be true according to the KB.**
- An inference algorithm is **complete** if it can derive **every conclusion that logically follows from the KB**. $KB \models \alpha \Rightarrow KB \vdash \alpha$
- In simple words: **If something is logically true according to the KB, the algorithm will be able to prove it.**

Propositional Logic → Inference Approaches

Category	Method	Core Question
General inference	**Model Checking**	"Is the query true in every model of KB?"
Rule-based inference	**Forward Chaining**	"What can I derive from the facts I already know?"
Rule-based inference	**Backward Chaining**	"What facts/rules do I need to prove this query?"
Proof-based inference	**Resolution**	"Can I derive a contradiction from KB + ¬Query?"
SAT-based inference	**DPLL**	"Can I find an assignment that satisfies the formula?"

Inference through Model Checking

- Suppose our Knowledge Base (KB) contains a couple of facts:
- **If it rains, the ground is wet.**
- **It rains.**

Let: R = It rains W = The ground is wet

- So: KB=(R→W)∧R
- **Query:** W?
- We want to check whether:
- KB ⊨ W
- **Step 1: List all possible models**
- There are two symbols: R and W. Therefore, there are: 2^2=4 possible models.

Model	R	W	(R -> W)	KB
1	T	T	T	T
2	T	F	F	F
3	F	T	T	F
4	F	F	T	F

Remember: KB=(R→W)∧R. So **KB is true only in Model 1**.

Step 2: Check the query

Our query is: W

In the only model where **KB is true**: W=T

Therefore: KB⊨W So, **the query W is entailed by the knowledge base.**

In simple words

Model checking means: Check all possible models and see whether the query is true in every model where the KB is true.

Inference through Model Checking - Drawback

- How can we check whether a sentence α is entailed by KB?

- we **enumerate all possible models of the KB** (truth assignments of all its symbols), and check that α is true in every model in which KB is true?
  - This is sound: All produced answer are correct.
  - This is complete: It will produce all correct answers.
  - **Problem**: if KB contains *n* symbols, the truth table will be of size 2^n

- Better idea: use ***inference rules***, or sound procedures to generate new sentences or *conclusions* given the *premises* in the KB.

Forward and Backward Chaining:

- Both standard forward and backward chaining can operate on a knowledge base of facts and definite/Horn rules. The difference is the direction in which they search/reason.
- So, let us first understand various types of inference rules and horn clause/rule, and so on.

Inference Rules

- Modus Ponens (Method of Supporting)

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

$\alpha \Rightarrow \beta, \alpha$ ← premises

β ← conclusion

This means: If the KB contains the sentences $\alpha \Rightarrow \beta$ and α then β is true.

- And-elimination

$$\frac{\alpha \wedge \beta}{\alpha}$$

Inference Rules

- And-introduction

$$\frac{\alpha,\ \beta}{\alpha \wedge \beta}$$

- Or-introduction

$$\frac{\alpha}{\alpha \vee \beta}$$

Inference Rules

- Double negative elimination

$$\frac{\neg\neg\alpha}{\alpha}$$

- Unit resolution

$$\frac{\alpha \vee \beta, \neg\beta}{\alpha}$$

Inference Rules

Modus Tollens (Method of Denying)

- Modus Tollens states that if an implication $P \rightarrow Q$ is true and its conclusion Q is false, then its premise P must be false.

$$\frac{P \rightarrow Q, \quad \neg Q}{\therefore \neg P}$$

What is a Literal and clause?

- A **literal** is either:
  - an atomic proposition, or its negation.
- Examples: P, ¬P, Q, ¬Q, R

- A **clause** is a disjunction (OR) of literals.
- Examples: P ∨ Q, ¬P ∨ R, P ∨ ¬Q ∨ R
- A single literal can also be considered a clause: P

Horn Clause and Rule

- A **Horn Clause** is simply a logical statement that has **at most one positive (true) conclusion**.
- The Horn **clause** is the OR form; the Horn **rule** is the convenient IF–THEN form used in Forward and Backward Chaining.

So:

$$\underbrace{\neg P \vee \neg Q \vee R}_{\text{Horn clause}}$$

is logically equivalent to:

$$\underbrace{P \wedge Q \rightarrow R}_{\text{Horn rule}}$$

Inference Rules

Multi-Premise (Horn Clause) Form:

- When applied to a Horn clause, if the conclusion is false, at least one of the premises must be false:

$$\frac{(P_1 \wedge P_2 \wedge \cdots \wedge P_n) \rightarrow Q, \quad \neg Q}{\therefore \neg(P_1 \wedge P_2 \wedge \cdots \wedge P_n) \equiv (\neg P_1 \vee \neg P_2 \vee \cdots \vee \neg P_n)}$$

- **Logical Rule:** $(P_1 \wedge P_2 \wedge \cdots \wedge P_n) \rightarrow Q$
- **Given:** $\neg Q$ (Q is False)
- **Inferred Conclusion:** At least one P_i is False

Resolution

$$\frac{\alpha \vee \beta, \neg\beta \vee \gamma}{\alpha \vee \gamma} \quad \text{or} \quad \frac{\alpha \vee \beta, \beta \Rightarrow \gamma}{\alpha \vee \gamma}$$

- Example:
  - α: “The weather is dry”
  - β: “The weather is rainy”
  - γ: “I carry an umbrella”

Important Inference rules helpful for Forward and Backward chaining

Rule	Pattern	Main idea
Modus Ponens	$P \rightarrow Q, P \Longrightarrow Q$	Confirm P
Modus Tollens	$P \rightarrow Q, \neg Q \Longrightarrow \neg P$	Reject Q
Hypothetical Syllogism	$P \rightarrow Q, Q \rightarrow R \Longrightarrow P \rightarrow R$	Join implications
Disjunctive Syllogism	$P \vee Q, \neg P \Longrightarrow Q$	Eliminate one possibility
Resolution	$PVQ, \neg PVR \Longrightarrow QVR$	Resolve complementary literals

Forward Chaining or Reasoning

- Forward chaining is **data-driven reasoning**.
- It starts from what we already know: FACTS and moves forward by applying rules.

Example: Knowledge Base (KB):

Facts:

- Fever
- Cough
- BreathingDifficulty

Rules:

- Fever $\wedge$ Cough $\rightarrow$ Infection
- Infection $\wedge$ BreathingDifficulty $\rightarrow$ Pneumonia
- Pneumonia $\rightarrow$ ImmediateAttention

Suppose the question is: Does the patient require immediate attention? (Q)

- **Our goal is:** ImmediateAttention (KB$\models$Q ?)

- **Forward Chaining — Step-by-Step Example**
- **Step 1**
  Known: Fever, Cough
  Rule: Fever ∧ Cough → Infection
  Therefore: Infection ✓

- **Step 2**
  Known: Infection, Breathing Difficulty
  Rule: Infection ∧ Breathing Difficulty → Pneumonia
  Therefore: Pneumonia ✓

- **Step 3**
  Known: Pneumonia
  Rule: Pneumonia → Immediate Attention
  Therefore: Immediate Attention ✓
- **Final Conclusion: Immediate Attention**

Forward Chaining Flow

FACTS

↓

Match rules

↓

Fire applicable rule

↓

Generate new fact

↓

Match rules again

↓

Generate new fact

↓

...

That's why it is called **forward** chaining.

- We move: **Known facts → New facts → New facts → Conclusion**

Backward Chaining

- Backward chaining works in the opposite direction.
- It is **goal-driven reasoning**.
- Suppose the question is: Does the patient require immediate attention?
- Our goal is: ImmediateAttention

Backward Chaining — Step-by-Step

Goal: ImmediateAttention

↓

Step 1 — Find a rule producing the goal

Pneumonia → ImmediateAttention

New Goal: Pneumonia

↓

Step 2 — Find a rule producing Pneumonia

Infection ∧ BreathingDifficulty → Pneumonia

New Goals: Infection **AND** BreathingDifficulty

↓

Step 3 — Check the known facts

BreathingDifficulty ✓ Already known

Need to prove: **Infection**

↓

Step 4 — Find a rule producing Infection

Fever ∧ Cough → Infection

New Goals: Fever **AND** Cough

↓

Step 5 — Check the known facts

Fever ✓
Cough ✓

Therefore: **Infection** ✓

Therefore, all subgoals are satisfied → Goal proven ✓

Resolution as an inference algorithm

- Forward/Backward Chaining works naturally with IF–THEN rules, while Resolution works with clauses; therefore, before applying Resolution, we convert the knowledge into CNF (**Conjunctive Normal Form)**.
- To prove a conclusion using Resolution, add the negation of the conclusion to the knowledge base and derive a contradiction (empty clause).

Resolution Inference Procedure

$$\frac{\alpha \vee \beta, \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- To prove KB $\models \alpha$, assume KB $\wedge \neg \alpha$ and derive a contradiction
- Rewrite KB $\wedge \neg \alpha$ as a conjunction of *clauses*, or disjunctions of *literals*
  - *Conjunctive normal form* (CNF)
- Keep applying resolution to clauses that contain *complementary literals* and adding resulting clauses to the list
  - If there are no new clauses to be added, then KB does not entail α
  - If two clauses resolve to form an *empty clause*, we have a contradiction and KB $\models \alpha$

What is CNF?

- CNF stands for: **Conjunctive Normal Form**
- The important thing is to understand the structure.
- **CNF = AND of ORs**
- For example:
  - $(P \vee Q) \wedge (R \vee S)$ - This is CNF.
- $(P \vee \neg Q \vee R) \wedge (S \vee T) \wedge (\neg U)$ - This is also CNF.

- CNF means: AND between clauses, OR inside each clause (disjunction (OR) of literals).

General Algorithm: Convert Knowledge into CNF

- **CNF = Conjunctive Normal Form, CNF is an AND of OR clauses.**

General structure: $(A \vee B) \wedge (C \vee D) \wedge (\neg E \vee F)$

The algorithm is:

Step 1: Eliminate $\leftrightarrow$

↓

Step 2: Eliminate $\rightarrow$

↓

Step 3: Move NOT inward

↓

Step 4: Apply De Morgan's laws

↓

Step 5: Distribute OR over AND

↓

Step 6: Simplify

↓

CNF

Some Examples

- **1: Eliminate Biconditional (↔)**

Whenever you have: $P \leftrightarrow Q$ replace it with: (P→Q)∧(Q→P)

- **2: Eliminate Implication (→):** Use this rule: P→Q≡¬P∨Q
- **3: Move NOT Inward**
- We want NOT (¬) to appear **only directly before propositions**.
  - **De Morgan's Law 1:** ¬(P∧Q)≡¬P∨¬Q
  - **De Morgan's Law 2:** ¬(P∨Q)≡¬P∧¬Q
- **4: Distribute OR over AND**
  - P∨(Q∧R)≡(P∨Q)∧(P∨R) or (P∧Q)∨R≡(P∨R)∧(Q∨R)

Our Previous medical example for Resolution

- **Given Query:** $Q = ImmediateAttention$
- For resolution, add the **negation of the conclusion**: $\neg ImmediateAttention$
- **KB + ¬Q:**
- **Facts:**
- Fever
- Cough
- BreathingDifficulty
- $\neg ImmediateAttention$← Negated conclusion
- **Rules converted to CNF:**
- $Fever \wedge Cough \rightarrow Infection$: $\boxed{\neg Fever \vee \neg Cough \vee Infection}$
- $Infection \wedge BreathingDifficulty \rightarrow Pneumonia$
  - $\boxed{\neg Infection \vee \neg BreathingDifficulty \vee Pneumonia}$
- $Pneumonia \rightarrow ImmediateAttention$
  - *¬Pneumonia ∨ ImmediateAttention*

CNF Clause Set:

$$C_1 : Fever$$

$$C_2 : Cough$$

$$C_3 : BreathingDifficulty$$

$$C_4 : \neg ImmediateAttention$$

$$C_5 : \neg Fever \vee \neg Cough \vee Infection$$

$$C_6 : \neg Infection \vee \neg BreathingDifficulty \vee Pneumonia$$

$$C_7 : \neg Pneumonia \vee ImmediateAttention$$

Resolution in step by step

- **Step 1 — Start with the negated conclusion**
- We want to prove: $ImmediateAttention$
- So assume the opposite: ¬ImmediateAttention

- **Step 2 — Resolve with the rule for ImmediateAttention**
- We have: C_7: $\neg ImmediateAttention$
- and C_6: $\neg Pneumonia \vee ImmediateAttention$
- The opposite literals are: $\neg ImmediateAttention$ and $ImmediateAttention$
- Cancel them.
- We get: ¬Pneumonia

- **Step 3 — Resolve with the rule for Pneumonia**
- We now have: $\neg Pneumonia$

and

- C_5: $\neg Infection \vee \neg BreathingDifficulty \vee Pneumonia$
- Cancel: $\neg Pneumonia$ and $Pneumonia$
- Result: ¬Infection∨¬BreathingDifficulty

- **Step 4 — Use the known fact BreathingDifficulty**
- We know: $BreathingDifficulty$
- Resolve:
- $\neg Infection \vee \neg BreathingDifficulty$
- with
- $BreathingDifficulty$
- Cancel: $\neg BreathingDifficulty$ and $BreathingDifficulty$
- Result: ¬Infection

- **Step 5 — Resolve with the rule for Infection**
- We have: $\neg Infection$

and

- C_4: $\neg Fever \vee \neg Cough \vee Infection$
- Cancel: $\neg Infection$ and $Infection$
- Result: ¬Fever∨¬Cough

- **Step 6 — Use the known fact Fever**
- We know: $Fever$
- Resolve:
- $\neg Fever \vee \neg Cough$
- With $Fever$
- Cancel:
- $\neg Fever$ and $Fever$
- Result: ¬Cough

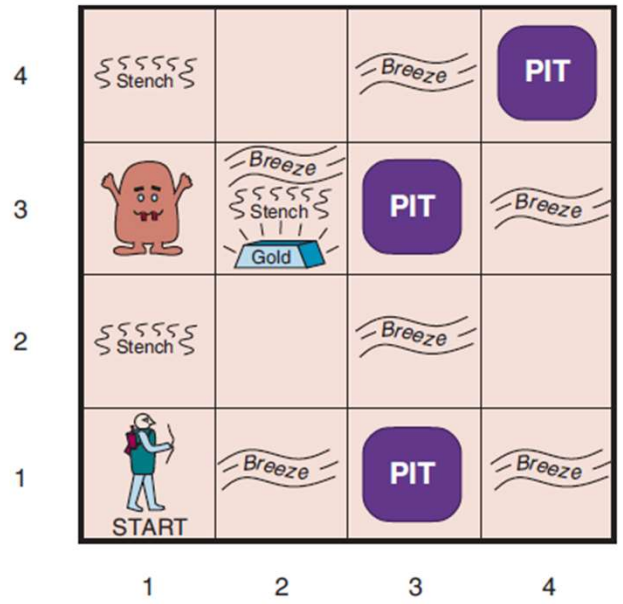
- **Step 7 — Use the known fact Cough**
- We know: $Cough$
- Resolve: $\neg Cough$

with

- $Cough$
- Cancel them: $\boxed{\square}$ The empty clause means **contradiction**.

- **Therefore, our assumption $\neg ImmediateAttention$ is false.**
- **Hence: KB ⊨ ImmediateAttention**

Example: Wumpus World



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

(a)

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(b)

Example: Wumpus World

Initial KB needs to contain rules like these for each square:

$Breeze(1,1) \Leftrightarrow Pit(1,2) \vee Pit(2,1)$
$Breeze(1,2) \Leftrightarrow Pit(1,1) \vee Pit(1,3) \vee Pit(2,2)$
$\text{Stench}(1,1) \Leftrightarrow W(1,2) \vee W(2,1)$
…

We have to enumerate all possible scenarios in propositional logic! First-order logic can help.

Percepts at (1,1) are no breeze or stench. Add the following facts to the KB:

$\neg Breeze(1,1)$
$\neg Stench(1,1)$

Inference will tell us that the following facts are entailed:

$\neg Pit(1,2), \neg Pit(2,1), \neg W(1,2), \neg W(2,1)$

This means that $(1,2)$ and $(2,1)$ are safe.

Summary

- Logical agents apply inference to a knowledge base to derive new information and make decisions.
- Basic concepts of logic:
  - syntax: formal structure of sentences
  - semantics: truth of sentences in models
  - entailment: necessary truth of one sentence given another
  - inference: deriving sentences from other sentences
  - soundness: derivations produce only entailed sentences
  - completeness: derivations can produce all entailed sentences
- Resolution is complete for propositional logic.
- Algorithms use forward, backward chaining, are linear in time, and complete for special clauses (definite clauses).

Limitations of Propositional Logic

Suppose you want to say “All humans are mortal”

- In propositional logic, you would need ~6.7 billion statements of the form:

  Michael_Is_Human and Michael_Is_Mortal,

  Sarah_Is_Human and Sarah_Is_Mortal, …

Suppose you want to say “Some people can run a marathon”

- You would need a disjunction of ~6.7 billion statements:

  Michael_Can_Run_A_Marathon or … or Sarah_Can_Run_A_Marathon

First-Order Logic

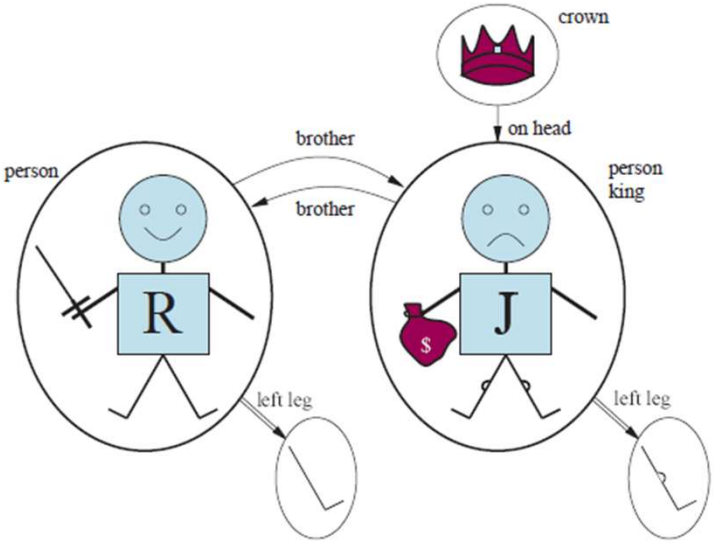
First-Order Logic

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

First-order Logic adds **objects** and **relations** to the facts of propositional logic.

This addresses the issues of propositional logic, which needs to store a fact for each instance of and object individually.

Syntax of FOL



$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \\
&\mid \neg\ \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
&\mid \textit{Quantifier}\ \textit{Variable}, \ldots\ \textit{Sentence} \\
\\
\textit{Term} &\rightarrow \textit{Function}(\textit{Term}, \ldots) \\
&\mid \textit{Constant} \\
&\mid \textit{Variable} \\
\\
\textit{Quantifier} &\rightarrow \forall \mid \exists \\
\textit{Constant} &\rightarrow A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} &\rightarrow a \mid x \mid s \mid \cdots \\
\textit{Predicate} &\rightarrow \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} &\rightarrow \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Objects

Relations. Predicate is/returns True or False

Function returns an object

FOL has five important building blocks: Objects + Variables + Predicates + Functions + Quantifiers

- **What exactly is an object?:** In FOL, an **object** is an individual/entity in our domain.
- In our university domain, objects could be: Ravi , Sita , Kumar , Meena , ComputerScience , Physics , AI , University

For example:

- Ravi is a student.
- Ravi is an object.
- Kumar is a professor.
- Kumar is an object.
- AI is a course.
- AI is an object.

- Usually, we represent a named object using a **constant**.

- **What is a predicate?:** A **predicate tells us something about an object or objects.**
- Think:
- **Predicate = property or relationship**
- There are two very useful categories.
- **A. Property:** Something about **one object**.
- Example: Ravi is a student - $Student(Ravi)$ -Student describes Ravi.
- Another:
- Kumar is a professor – $Professor(Kumar)$
- Another: AI is difficult - $Difficult(AI)$
- These are **one-place predicates**.

- **Relationship**
- Now consider: Ravi studies Computer Science.
- Here there are **two entities**:
- Ravi , Computer Science and there is a relationship: Ravi **studies** Computer Science.
- So: $Studies(Ravi, ComputerScience)$
- Here:
- Ravi → object
- ComputerScience → object
- Studies → relationship/predicate

- **The most important distinction**

Look at these two:

- **Sentence 1**
- Ravi is a student. $Student(Ravi)$**.** Student is a **property**.
- **Sentence 2**
- Ravi studies Computer Science.
- $Studies(Ravi, ComputerScience)$
- Studies represents a **relationship between two objects**.
- So:
- **Predicate can represent either a property or a relationship.**

One-place predicate
Suppose: Ravi is a student.
Ask: How many objects are involved?
Only Ravi.
Therefore:
Student(Ravi)

- **More examples**
- Sita is a student. $Student(Sita)$
- Kumar is a professor. $Professor(Kumar)$
- Meena is faculty. $Faculty(Meena)$
- AI is difficult. $Difficult(AI)$
- Ravi is intelligent. $Intelligent(Ravi)$
- All are: $\boldsymbol{Predicate(object)}$

Two-place predicate
Now:
Ravi studies Computer Science.
Two objects: Ravi , Computer Science
Relationship: studies
Therefore:
Studies(Ravi,ComputerScience)

- **More examples**
- Kumar teaches AI. $Teaches(Kumar, AI)$
- Ravi attends Mathematics. $Attends(Ravi, Mathematics)$
- Kumar advises Ravi. $Advises(Kumar, Ravi)$
- Ravi likes Sita. $Likes(Ravi, Sita)$
- General pattern: $\boldsymbol{Relationship}(\boldsymbol{Object1}, \boldsymbol{Object_2})$

Order of objects matters

- This is **very important**.
- Consider:
- Kumar teaches Ravi. $Teaches(Kumar, Ravi)$
- This does **not** mean: Ravi teaches Kumar.
- That would be: $Teaches(Ravi, Kumar)$
- Similarly:
- $Advises(Kumar, Ravi)$
- means: Kumar advises Ravi.
- Not:
- Ravi advises Kumar.
- So when translating a relationship, ask: **Who does what to whom?**

- **Three-place predicates**
- Some relationships involve three objects.
- Suppose: Kumar teaches AI to Ravi.
- We could represent: $Teaches(Kumar, AI, Ravi)$
- Three objects: Kumar , AI , Ravi
- Predicate: Teaches
- So:
- $\boldsymbol{Predicate(Object1, Object2, Object_3)}$
- Another example: Ravi submits Assignment1 to Kumar.
- Submits(Ravi,Assignment1,Kumar)

Constants vs variables

- Now we introduce variables.
- Suppose: Ravi is a student.
- We know exactly who.
- So:
- $Student(Ravi)$
- But suppose:
- Someone is a student. We don't know who.
- We use a variable: x
- Read: There exists someone who is a student.
- **Variable = placeholder:** Think of x as an empty box.
- $Student(x)$ means: x is a student.
- The x could eventually become: $Ravi$, or: $Sita$ or: $Kumar$
- So: $Student(x)$ is general. While: $Student(Ravi)$ is specific.

Functions

- Consider: Ravi's advisor is Kumar.
- We could represent the advisor relationship using a predicate: $Advisor(Ravi, Kumar)$
- But suppose we want to talk about **the advisor of Ravi as an object**.
- We can use a function: $AdvisorOf(Ravi)$. This function returns an individual.
- Then: Ravi's advisor is a professor.
- could be:
- $Professor = \left(AdvisorOf(Ravi)\right)$

Here:

- Ravi → constant
- AdvisorOf → function
- Professor → predicate

Predicate vs function

- **Predicate:** Answers something like: Is this true?
- Example: $Student(Ravi)$
- It tells us: Ravi is a student.
- **Function**
- Returns an object- $AdvisorOf(Ravi)$
- It means: the advisor associated with Ravi.
- Then we can say:
- $Professor = \left(AdvisorOf(Ravi)\right)$
- Meaning:
- Ravi's advisor is a professor.
- **simple memory trick**

- **Predicate → describes/asserts and Function → returns an object**

Quantifiers

- There are two main quantifiers: **Universal quantifier:** ∀
- Read as: **for all** / **for every / for each**
- **Existential quantifier:** ∃
- Read as: **there exists** / **there is at least one / some**

- **Statement 1** Everyone is intelligent.
- $\forall x \ Intelligent(x)$ Means: **Every individual is intelligent.**

- **Statement 2** Someone is intelligent.
- $\exists x \ Intelligent(x)$ Means: **At least one individual is intelligent.**
- These are completely different.

- Suppose we say: All humans are mortal.
- FOL: $\forall x \ \big(Human(x) \rightarrow Mortal(x)\big)$
- Read: For every x, if x is human, then x is mortal.

- Suppose we say: Some human is intelligent.
- We can represent: $\exists x \ \big(Human(x) \wedge Intelligent(x)\big)$
- Read:
- There exists at least one x such that x is human and x is intelligent.
- We don't know who that person is. It could be Ravi. It could be Sita.
- It could be John. We only know that **at least one exists**.

Properties of Quantifiers

- $\forall$**x** $\forall$**y** is the same as $\forall$**y** $\forall$**x**
- $\exists$**x** $\exists$**y** is the same as $\exists$**y** $\exists$**x**
- $\exists$**x** $\forall$**y** is not the same as $\forall$**y** $\exists$**x**

  $\exists$**x** $\forall$**y Loves(x,y)**

  "There is a person who loves everyone"

  $\forall$**y** $\exists$**x Loves(x,y)**

  "Everyone is loved by at least one person"

- **Quantifier duality:** each quantifier can be expressed using the other with the help of negation

  $\forall$**x Likes(x,IceCream)** $\neg\exists$**x**

  $\exists$**x Likes(x,Broccoli)** $\neg\forall$**x**

Concept	Example	Meaning
Object/entity	Ravi	A particular individual
Constant	Ravi	Symbol naming an object
Variable	x	Represents an unspecified object
Predicate/property	Student(x)	x has property student
Relationship	Studies(x,AI)	x studies AI
Function	AdvisorOf(x)	returns x's advisor
Universal quantifier	∀x	for every x
Existential quantifier	∃x	there exists an x
AND	∧	both
OR	∨	either/both
NOT	¬	not
Implication	→	if...then

Practice examples for English to FOL

1. Ravi is a student. Answer: $\boxed{Student(Ravi)}$

2. Sita is a professor. $\boxed{Professor(Sita)}$

3. Kumar is intelligent. $\boxed{Intelligent(Kumar)}$

4. AI is a course. $\boxed{Course(AI)}$

5. Ravi studies AI. $\boxed{Studies(Ravi, AI)}$

6. Kumar teaches AI. $\boxed{Teaches(Kumar, AI)}$

7. Sita studies Physics. $\boxed{Studies(Sits, Physics)}$

8. Kumar advises Ravi. $\boxed{Advises(Kumar, Ravi)}$

9. Ravi attends Mathematics. $\boxed{Attends(Ravi, Mathematics)}$

10. Ravi is intelligent and Ravi studies AI.

Intelligent(Ravi)∧Studies(Ravi,AI)

11. Every student is a learner.

- Start with: Every → $\forall x$
- Student: $Student(x)$
- Learner: $Learner(x)$
- "If student, then learner": ∀x(Student(x)→Learner(x))

12. Every professor is faculty. $\boxed{\forall x\big(Professor(x) \rightarrow Faculty(x)\big)}$

13. Every student studies AI.

- $\boxed{\forall x\big(Student(x) \rightarrow Studies(x, AI)\big)}$
- Notice that **AI is a constant**, not a variable.

- **14. Every professor teaches AI.**
- $\boxed{\forall x\big(Professor(x) \rightarrow Teaches(x, AI)\big)}$

- **15. Every student is intelligent.**
- $\boxed{\forall x\big(Student(x) \rightarrow Intelligent(x)\big)}$

- **16. Every course is difficult.**
- ∀x(Course(x)→Difficult(x))

- **17. Every student attends a university.**
- Assuming University is the name of our university:
- $\boxed{\forall x\big(Student(x) \rightarrow Attends(x, University)\big)}$

- **18. Every professor teaches a course.**
- Here "a course" means **some course**, not one specific named course.
- So we need another variable and ∃.
- $\boxed{\forall x\left(Professor(x) \rightarrow \exists y\big(Course(y) \wedge Teaches(x, y)\big)\right)}$
- Read: For every professor x, there exists a course y such that x teaches y.

- **19. Some student is intelligent.** $\boxed{\exists x\big(Student(x) \wedge Intelligent(x)\big)}$
- **20. Some professor teaches AI.** $\boxed{\exists x\big(Professor(x) \wedge Teaches(x, AI)\big)}$
- **21. Some student studies AI.** $\boxed{\exists x\big(Student(x) \wedge Studies(x, AI)\big)}$
- **22. There is a student who studies Physics.** $\boxed{\exists x\big(Student(x) \wedge Studies(x, Physics)\big)}$
- **23. There is a professor who teaches Physics.**
- $\boxed{\exists x\big(Professor(x) \wedge Teaches(x, Physics)\big)}$

- **24. Some students are intelligent.**
- In basic FOL, "some" means **at least one**: $\boxed{\exists x\big(Student(x) \wedge Intelligent(x)\big)}$

- **25. At least one professor teaches AI.** *∃x(Professor(x)∧Teaches(x,AI))*

- **26. Every student who studies AI is intelligent.**
- Identify: Student: $Student(x)$
- Studies AI: $Studies(x, AI)$
- Both conditions: $Student(x) \wedge Studies(x, AI)$
- Conclusion: $Intelligent(x)$
- Therefore:
- $\boxed{\forall x \left(\left(Student(x) \wedge Studies(x, AI) \right) \rightarrow Intelligent(x) \right)}$

- **27. Every professor who teaches AI is faculty.**
- $\boxed{\forall x \left(\left(Professor(x) \wedge Teaches(x, AI) \right) \rightarrow Faculty(x) \right)}$
-

- **28. Every student who studies Physics is a Physics student.**
- $\boxed{\forall x \left(\left(Student(x) \wedge Studies(x, Physics) \right) \rightarrow PhysicsStudent(x) \right)}$
-

- **29. Every student who studies AI and Physics is intelligent.**
- $\boxed{\forall x \left(\left(Student(x) \wedge Studies(x, AI) \wedge Studies(x, Physics) \right) \rightarrow Intelligent(x) \right)}$
- There are **three conditions** before the implication.

- **30. No student is a professor.**
- "No" can be represented using negation:
- $\boxed{\forall x\big(Student(x) \rightarrow \neg Professor(x)\big)}$
- Meaning: For every x, if x is a student, x is not a professor.
- **31. No professor is a student.**
- $\boxed{\forall x\big(Professor(x) \rightarrow \neg Student(x)\big)}$

- **32. Some student is not intelligent.**
- $\boxed{\exists x\big(Student(x) \wedge \neg Intelligent(x)\big)}$
- Notice:
- **Some →** $\exists$
- **not intelligent →** $\neg Intelligent(x)$

- **33. Every student studies AI or Physics.**
- $\boxed{\forall x \left(Student(x) \rightarrow \left(Studies(x, AI) \vee Studies(x, Physics) \right) \right)}$
-

- **34. No student studies both AI and Physics.**
- ∀x(Student(x)→¬(Studies(x,AI)∧Studies(x,Physics)))

- **35. Every student studies at least one course.**
- First: Every student → $\forall x$
- A course → $\exists y$
- Relationship → Studies
- Therefore:
- $\boxed{\forall x \left(Student(x) \rightarrow \exists y \big(Course(y) \wedge Studies(x, y) \big) \right)}$
- Read it slowly:
- For every student x, there exists at least one course y that x studies.

- **36. Every professor teaches at least one course.**
- $\boxed{\forall x \left(Professor(x) \rightarrow \exists y \big(Course(y) \wedge Teaches(x, y) \big) \right)}$
-

- **37. Every course is studied by at least one student.**
- $\boxed{\forall y \left(Course(y) \rightarrow \exists x \big(Student(x) \wedge Studies(x, y) \big) \right)}$
- Notice the order:
- Every **course** → y
- and:
- at least one **student** → x

- **38. Every student has an advisor.**
- Let's represent Advisor(y) and Advises(y,x).
- $\boxed{\forall x \left(Student(x) \rightarrow \exists y \big(Advisor(y) \wedge Advises(y, x) \big) \right)}$
- Read: Every student x has some advisor y who advises x.
- **39. Every professor advises at least one student.**
- ∀x(Professor(x)→∃y(Student(y)∧Advises(x,y)))
- **40. There is a professor who advises every student.**
- This is very different from #39.
- "There is a professor" → $\exists x$
- "every student" → $\forall y$
- $\boxed{\exists x \left(Professor(x) \wedge \forall y \big(Student(y) \rightarrow Advises(x, y) \big) \right)}$
- Read: There exists one professor who advises every student.

The 5 questions you should ask every time?

Ask these five questions:

① WHO / WHAT?

Find the objects. Ravi studies AI. → Ravi, AI

② WHAT IS SAID ABOUT THEM?

Find the property/relationship. Studies → Studies

③ KNOWN OR UNKNOWN?

Ravi → constant

someone / every student → variable

④ EVERY OR SOME?

Every → ∀

Some / there exists → ∃

⑤ HOW ARE THE PARTS CONNECTED?

And → ∧

Or → ∨

Not → ¬

if...then → →

Unification

Unification is the process of making two FOL expressions identical by finding suitable substitutions for their variables.

The key word is: **MATCH**

For example: $Student(x)$ and $Student(Ravi)$

Can they match? Yes.

If we substitute: $\boxed{x = Ravi}$

then: $Student(x)$ becomes: $Student(Ravi)$

Now both expressions are identical.

That substitution is called a **unifier**.

A substitution means: Replace a variable with a term.

For example:

$x/Ravi$ means: Replace x with Ravi.

So:

$Student(x)$ after substitution: $Student(Ravi)$

Example 1

$Student(x)$ and $Student(Ravi)$

We need: $x = Ravi$

Therefore: $\boxed{\{x/Ravi\}}$ is the substitution.

After applying it: $Student(x)$ becomes $Student(Ravi)$

Example 2

$Professor(x)$ and $Professor(Kumar)$ Substitution:

$\boxed{\{x/Kumar\}}$ Therefore they unify.

Example 3 $Course(x)$ and $Course(AI)$

Substitution: $\boxed{\{x/AI\}}$

Why do we need Unification?

This is where it connects directly to **inference**.

Suppose our university KB contains:

Rule: Every professor is faculty. $\forall x\big(Professor(x) \rightarrow Faculty(x)\big)$

Fact: Kumar is a professor. $Professor(Kumar)$

We want to apply the rule.

The rule contains: $Professor(x)$

The fact contains: $Professor(Kumar)$

We need to determine:

What should x be? Unification says: $\boxed{x = Kumar}$

So the rule becomes: $Professor(Kumar) \rightarrow Faculty(Kumar)$

Since: $Professor(Kumar)$

is known, we derive: $Faculty(Kumar)$

So: **Unification helps us match a rule with a fact.**

The basic pattern

Whenever you see: $P(x)$ and: $P(A)$

you can usually unify them with: $\boxed{x = A}$

Examples: $Student(x)$ with: $Student(Ravi)$

$\rightarrow x = Ravi$

$Professor(x)$ with: $Professor(Meena)$

$\rightarrow x = Meena$

$Course(x)$ with: $Course(AI)$

→ x=AI

Unification is NOT just matching names

Consider: $Studies(x, AI)$ and: $Studies(Ravi, AI)$

We compare argument by argument.

Second arguments already match.

First arguments require: $x = Ravi$

Therefore: {x/Ravi}

Another example: $Teaches(x, AI)$ and: $Teaches(Kumar, AI)$

So: {x/Kumar}

Multiple variables

Consider: $Studies(x, y)$ and: $Studies(Ravi, AI)$

Compare each position:

First argument $x = Ravi$

Second argument $y = AI$

Therefore: $\boxed{\{x,/Ravi\ \ y/AI\}}$

After substitution: $Studies(Ravi, AI)$

Another multiple-variable example

$Teaches(x, y)$ and: $Teaches(Kumar, AI)$

Therefore: {x/Kumar,y/AI}

Variables can match variables

Consider: $Studies(x, AI)$

and: $Studies(y, AI)$

We can make them match by: $x = y$

For example: $\boxed{\{x/y\}}$

or equivalently, depending on convention: $\boxed{\{y/x\}}$

The important idea is that **variables can be unified with other variables**.

Constants cannot normally be changed

Consider: $Student(Ravi)$ and: $Student(Sita)$

Can we unify them? No.

Why?

Because:

$$Ravi \neq Sita$$

Both are constants.

We cannot substitute Ravi with Sita.

Therefore: FAIL

Predicate names must match

Consider: $Student(x)$ and: $Professor(Ravi)$

Can they unify?No.

Because:

$$Student \neq Professor$$

Therefore: FAIL

Number of arguments must match

Consider: $Studies(x, AI)$ and: $Studies(Ravi)$

Cannot unify. Why?

First has **2 arguments**.

Second has **1 argument**.

Therefore: FAIL

Argument order matters:

Consider: $Teaches(x, AI)$ and: $Teaches(AI, Physics)$

Compare:

Teaches(x , AI)

Teaches(AI , Physics)

First position: $x = AI$

But second position requires:

$$AI = Physics$$

which is false. Therefore: FAIL

One mismatch can make the entire unification fail.

Unification algorithm

When you need to unify two expressions, follow these steps.

Step 1 Check the **predicate/function name**. Must be the same.

Step 2 Check the **number of arguments**. Must be the same.

Step 3 Compare arguments from left to right.

Step 4

If: $variable + constant$ → substitute the variable.

Step 5

If: $variable + variable$ → make them equal.

Step 6 If two different constants occur:

$Ravi \neq Sita$ → FAIL.

Step 7

Check that all substitutions are consistent.

Let's practice

Q1 Unify: $Student(x)$ and: $Student(Ravi)$

Answer $\boxed{\{x/Ravi\}}$

Q2 Unify: $Studies(x, AI)$ and: $Studies(R, aviAI)$

Answer $\boxed{\{x/Ravi\}}$

Q3 Unify: $Studies(x, y)$ and: $Studies(R, aviAI)$

Answer $\boxed{\{x,/Ravi\ y/AI\}}$

Q4 Unify: $Student(Ravi)$ and: $Student(Sita)$

Answer FAIL

Q5 Unify: $Student(x)$ and: $Professor(Ravi)$ **Answer** $\boxed{\text{FAIL}}$

Q6 Unify: $Teaches(x, AI)$ and: $Teaches(K, umarPhysics)$

First: $x = Kumar$ Second requires: $AI = Physics$

Impossible. Therefore: $\boxed{\text{FAIL}}$

Expression 1	Expression 2	Result
Student(x)	Student(Ravi)	✅ {x/Ravi}
Student(x)	Student(Priya)	✅ {x/Priya}
Student(Ravi)	Teacher(Ravi)	❌ Different predicates
Studies(x,AI)	Studies(Ravi,AI)	✅ {x/Ravi}
Studies(x,y)	Studies(Ravi,AI)	✅ {x/Ravi,y/AI}
Studies(x,x)	Studies(Ravi,AI)	❌ x needs two different values
Teaches(x,AI)	Teaches(DrRavi,AI)	✅ {x/DrRavi}
Teaches(x,AI)	Studies(DrRavi,AI)	❌ Different predicates
Enrolled(x,AI)	Enrolled(x,DBMS)	❌ AI ≠ DBMS
Likes(x,x)	Likes(Ravi,Ravi)	✅ {x/Ravi}
Likes(x,x)	Likes(Ravi,Priya)	❌ x needs both Ravi and Priya

Forward and Backward Chaining in First-Order Logic (FOL).

- **Forward chaining = Start with known facts → apply rules → derive new facts → continue until the required conclusion is obtained.**
- So the direction is: **Facts → Rules → New Facts → More New Facts → Conclusion**
- Think of it as **reasoning forward from what we already know**.

- **Forward = "What can I conclude from what I know?"**
- **Backward = "What do I need to know to prove my goal?"**

Example 1 — Student and Scholarship

Knowledge Base

- **Facts:**
- $Student(Ravi)$
- $GoodMarks(Ravi)$
- $Regular(Ravi)$
- **Rules:**
- **R1:** $Student(x) \land GoodMarks(x) \rightarrow Eligible(x)$
- **R2:** $Eligible(x) \land Regular(x) \rightarrow Scholarship(x)$

Query

- Is Ravi eligible for a scholarship?

- Forward chaining starts with **facts**.

Step 1

- We know: Student(Ravi), GoodMarks(Ravi), Regular(Ravi)
- R1 says: $Student(x) \wedge GoodMarks(x) \rightarrow Eligible(x)$
- As per Unification, For x = Ravi:
  - Student(Ravi) ✓ GoodMarks(Ravi) ✓
- Therefore: $Eligible(Ravi)$ is derived.

Step 2

- Now we have: Eligible(Ravi), Regular(Ravi)
- Apply R2: $Eligible(x) \wedge Regular(x) \rightarrow Scholarship(x)$
- Therefore: Scholarship(Ravi)

B. Backward Chaining: Backward chaining starts with the goal.

Our goal is: $Scholarship(Ravi)$

Ask: Which rule can produce Scholarship(Ravi)?

R2: $Eligible(x) \wedge Regular(x) \rightarrow Scholarship(x)$

Therefore, we need to prove: Eligible(Ravi) and Regular(Ravi)

Regular(Ravi) is already a fact. ✓

Now we need: $Eligible(Ravi)$

Which rule produces it?

R1: $Student(x) \wedge GoodMarks(x) \rightarrow Eligible(x)$

Therefore we need: *Student(Ravi), GoodMarks(Ravi)*

Both are facts. ✓

Therefore: Eligible(Ravi) ✓ ↓Scholarship(Ravi) ✓

Example 2 — University Course Eligibility

Facts

- *Student(Anita)*
- *Completed(Anita,Programming)*
- *Completed(Anita,Mathematics)*
- *Passed(Anita,Programming)*

Rules

- **R1:** A student who completed Programming and Mathematics has the prerequisites for AI.
- $Completed(x, Programming) \land Completed(x, Mathematics) \rightarrow HasPrerequisites(x, AI)$
- **R2:** A student with AI prerequisites and who passed Programming can register for AI.
- $HasPrerequisites(x, AI) \land Passed(x, Programming) \rightarrow CanRegister(x, AI)$
- **R3:** A student who can register for AI and is a student is an AI-course candidate.
- $CanRegister(x, AI) \land Student(x) \rightarrow AICandidate(x)$

Query

- Is Anita an AI-course candidate?

A. Forward Chaining

Start with:

Student(Anita)

Completed(Anita, Programming)

Completed(Anita, Mathematics)

Passed(Anita, Programming)

Step 1 — R1

We have: Completed(Anita, Programming) ✓

Completed(Anita, Mathematics) ✓

Therefore: $HasPrerequisites(A, nitaAI)$

Step 2 — R2

We now have: *HasPrerequisites(Anita, AI)* ✓

Passed(Anita, Programming) ✓

Therefore: $CanRegister(A, nitaAI)$

Step 3 — R3

We have: CanRegister(Anita, AI) ✓

Student(Anita) ✓

Therefore: AICandidate(Anita)

B. Backward Chaining

Start with:

$$AICandidate(Anita)$$

Which rule produces this?

R3: $CanRegister(x, AI) \wedge Student(x) \rightarrow AICandidate(x)$

So we need:

CanRegister(Anita, AI)

Student(Anita)

Student(Anita) is already a fact. ✓

Now prove: CanRegister(Anita,AI)

R2 can produce it:

$$HasPrerequisites(x, AI) \wedge Passed(x, Programming) \rightarrow CanRegister(x, AI)$$

So we need:

HasPrerequisites(Anita, AI)

Passed(Anita, Programming)

Passed(Anita,Programming) is a fact. ✓

Now prove:

HasPrerequisites(Anita,AI)

R1 produces it:

$$Completed(x, Programming) \land Completed(x, Mathematics) \rightarrow HasPrerequisites(x, AI)$$

We have both facts. ✓

Therefore:

HasPrerequisites(Anita, AI) ✓

↓

CanRegister(Anita, AI) ✓

↓

AICandidate(Anita) ✓

Example 3 — Professor, Research and Project

- **Facts**
- Professor(DrRao)
- Researches(DrRao,AI)
- Student(Ravi)
- Student(Priya)
- Studies(Ravi,AI)
- Studies(Priya,ML)
- Interested(Ravi,Research)
- **Rules**
- **R1** A professor who researches AI supervises AI projects.
- $Professor(x) \land Researches(x, AI) \rightarrow Supervises(x, AIProject)$
- **R2** A student who studies AI is an AI learner.
- $Student(x) \land Studies(x, AI) \rightarrow AILearner(x)$
- **R3** An AI learner who is interested in research is a research candidate.
- $AILearner(x) \land Interested(x, Research) \rightarrow ResearchCandidate(x)$
- **R4** A research candidate can apply for an AI project.
- $ResearchCandidate(x) \rightarrow CanApply(x, AIProject)$
- **Query** Can Ravi apply for an AI project?

A. Forward Chaining

- Start with all facts:
- Professor(DrRao)
- Researches(DrRao, AI)
- Student(Ravi)
- Student(Priya)
- Studies(Ravi, AI)
- Studies(Priya, ML)
- Interested(Ravi, Research)

Step 1 — Apply R1

- Professor(DrRao) ✓
- Researches(DrRao, AI) ✓
- Therefore: $Supervises(DrRao, AIProject)$
- This fact is not needed for our current query, but forward chaining may still derive it.

- **Step 2 — Apply R2 to Ravi**
- Student(Ravi) ✓
- Studies(Ravi, AI) ✓
- Therefore:
- $AILearner(Ravi)$

- **Step 3 — Apply R2 to Priya**
- We have:
- Student(Priya) ✓
- Studies(Priya, ML) ✓
- But R2 requires:
- Studies(Priya, AI)
- We don't have that.
- Therefore:
- ❌ No AILearner(Priya).

- **Step 4 — Apply R3**
- We now have:
- *AILearner(Ravi)*
- *Interested(Ravi, Research)*
- Therefore: $ResearchCandidate(Ravi)$

- **Step 5 — Apply R4**
- We have:
- $ResearchCandidate(Ravi)$
- Therefore: $CanApply(Ravi, AIProject)$
- **Final result**
- $\boxed{CanApply(Ravi, AIProject)}$ is proved.

B. Backward Chaining

- We start with the goal:
- $CanApply(Ravi, AIProject)$
- **Step 1**
- Which rule produces CanApply?
- R4:
- $ResearchCandidate(x) \rightarrow CanApply(x, AIProject)$
- So our new goal becomes: *ResearchCandidate(Ravi)*

- **Step 2**
- Which rule produces ResearchCandidate?
- R3:
- $AILearner(x) \wedge Interested(x, Research) \rightarrow ResearchCandidate(x)$
- Therefore we need:
- AILearner(Ravi) and Interested(Ravi, Research)
- Interested(Ravi,Research) is already a fact. ✓
- We still need: $AILearner(Ravi)$

- **Step 3**
- Which rule produces AILearner?
- R2: $Student(x) \wedge Studies(x, AI) \rightarrow AILearner(x)$
- Therefore we need: Student(Ravi) and Studies(Ravi, AI)
- Both are facts. ✓
- Therefore:

AILearner(Ravi) ✓

↓

ResearchCandidate(Ravi) ✓

↓

CanApply(Ravi, AIProject) ✓

Resolution in First-Order Logic (FOL)

- **Resolution is a rule of inference used to prove whether a conclusion logically follows from a set of FOL statements.**
- The basic idea is: **To prove a statement, assume its opposite and try to derive a contradiction.** This is called **proof by contradiction**.
- Resolution cannot directly work with rules such as:

$$Student(x) \rightarrow Studies(x)$$

- We first need to convert FOL statements into a special form called **Clause Form / CNF**.
- CNF means: **Conjunctive Normal Form : AND of ORs**
- The important structure is: $(A \vee B) \wedge (C \vee D)$
- That is: AND between groups. OR inside each group
- For example:
- $\big(Student(x) \vee Professor(x)\big) \wedge \big(Studies(x, AI) \vee Teaches(x, AI)\big)$ is in CNF.
- Each group separated by $\wedge$ is called a **clause**.

CNF conversion in FOL

- **The CNF conversion steps**

FOL sentence

↓

1. Remove →

↓

2. Move ¬ inward

↓

3. Standardize variables

↓

4. Skolemization

↓

5. Remove ∀

↓

6. Distribute ∨ over ∧

↓

CNF

Step 1 — Remove implication

- Remember: $A \rightarrow B$ becomes: $\boxed{\neg A \vee B}$
- **Example** $Student(x) \rightarrow Studies(x, AI)$ becomes: *¬Student(x) ∨Studies(x,AI)*

Step 2 — Move NOT inward using De Morgan's law.

- We use two important rules.
- **Rule 1** $\neg(A \wedge B)$ becomes: $\neg A \vee \neg B$
- **Rule 2** $\neg(A \vee B)$ becomes: $\neg A \wedge \neg B$
- Suppose:

$$\neg\big(Student(x) \wedge Studies(x, AI)\big)$$

- Then:

¬Student(x) ∨¬Studies(x,AI)

Step 3 — Standardize variables

- This becomes important when we have multiple statements containing variables.
- Suppose we have:

$$Student(x) \rightarrow Studies(x, AI)$$

- and

$$Professor(x) \rightarrow Teaches(x, AI)$$

- These x's belong to **different rules**.
- We can rename them:

$$Student(x) \rightarrow Studies(x, AI)$$
$$Professor(y) \rightarrow Teaches(y, AI)$$

- This avoids accidentally treating the two x^s as the same variable.
- So: **Standardization means giving different variables different names when necessary.**

Step 4 — Skolemization

- Skolemization creates a placeholder for an unknown existing object.
- **Skolemization means replacing an existential quantifier (∃) with a new constant or a new function.**
- In simple words:

  "There exists some object" → give that object a new name.

1. **∃ alone → Skolem constant:**

- Suppose: Some student is intelligent.

$$\exists x \ \big(Student(x) \wedge Intelligent(x)\big)$$

- We don't know who the student is, so give that unknown student a new name, say a:

$$\boxed{Student(a) \wedge Intelligent(a)}$$

- Here a is called a **Skolem constant**.

2. ∃ depending on ∀ → Skolem function

- Suppose: Everyone studies some course.

$$\forall x \exists y \; Studies(x, y)$$

- The course y can depend on which person x we are talking about.
- So replace:

$$y$$

with:

$$f(x)$$

giving:

$$\boxed{\forall x \; Studies(x, f(x))}$$

- Here f(x) is a **Skolem function**.

Another Example:

- For example:
- Every student studies **some course**.
- FOL:

$$\forall x \left(Student(x) \rightarrow \exists y \big(Course(y) \wedge Studies(x, y) \big) \right)$$

- The y is existential.
- We cannot simply remove ∃y.
- We replace it with a **Skolem function**.
- For example: $f(x)$
- So y becomes:

$$y = f(x)$$

- Giving:

$$Student(x) \rightarrow (Course(f(x)) \wedge Studies(x, f(x)))$$

So, Skolemization is the process of replacing 'there exists' with a suitable new name or function

- **Step 5 — Remove universal quantifiers**
- Once we have dealt with existential variables through Skolemization, we can drop ∀.
- For example:

$$\forall x\big(Student(x) \rightarrow Studies(x, AI)\big)$$

- After removing the implication:

$$\forall x\big(\neg Student(x) \vee Studies(x, AI)\big)$$

- We can write:

$$\boxed{\neg Student(x) \vee Studies(x, AI)}$$

Step 6 — Distribute OR over AND

Suppose we have: $A \vee (B \wedge C)$

This is **not CNF**.

We distribute OR: $(A \vee B) \wedge (A \vee C)$

Simple complete example

- Let's convert: Every student is hardworking.
- **Step 1: FOL**

$$\forall x\big(Student(x) \rightarrow Hardworking(x)\big)$$

- **Step 2: Remove implication**

$$\forall x\big(\neg Student(x) \vee Hardworking(x)\big)$$

- **Step 3: Remove ∀**

$$\boxed{\neg Student(x) \vee Hardworking(x)}$$

- This is CNF.
- So our clause is: *¬Student(x) ∨Hardworking(x)*

Another Example

- Sentence: Every student who studies AI is an AI learner.
- **FOL**

$$\forall x \Big(\big(Student(x) \land Studies(x, AI) \big) \rightarrow AILearner(x) \Big)$$

- **Step 1 — Remove implication**
- Use: $A \rightarrow B \equiv \neg A \lor B$
- So:

$$\neg \big(Student(x) \land Studies(x, AI) \big) \lor AILearner(x)$$

- **Step 2 — Apply De Morgan**

$$\big(\neg Student(x) \lor \neg Studies(x, AI) \big) \lor AILearner(x)$$

- Simplify:

$$\boxed{\neg Student(x) \lor \neg Studies(x, AI) \lor AILearner(x)}$$

- This is already a clause, so it is CNF.

Resolution Example

- **Example 1 — Student and Course**
- **Given facts**
- Ravi is a student. $Student(Ravi)$
- Ravi studies AI. $Studies(Ravi, AI)$
- **Rule**
- Every student who studies AI is an AI learner.

$$Student(x) \land Studies(x, AI) \rightarrow AILearner(x)$$

- **Query**
- Ravi is an AI learner.

AILearner(Ravi)

- **Step 1: Negate the query**
- For resolution, we assume the query is false.
- Query:

$$AILearner(Ravi)$$

- Negated query:

$$\boxed{\neg AILearner(Ravi)}$$

- Add this to our knowledge base.

- **Step 2: Convert the rule to CNF**
- We have:

$$Student(x) \land Studies(x, AI) \rightarrow AILearner(x)$$

- Remove →:

$$\neg\big(Student(x) \land Studies(x, AI)\big) \lor AILearner(x)$$

- Apply De Morgan's law:

$$\boxed{\neg Student(x) \lor \neg Studies(x, AI) \lor AILearner(x)}$$

- This is our clause.

- **Step 3: List all clauses**
- We now have:
- **Clause 1**

$$Student(Ravi)$$

- **Clause 2**

$$Studies(Ravi, AI)$$

- **Clause 3 — Rule**

$$\neg Student(x) \vee \neg Studies(x, AI) \vee AILearner(x)$$

- **Clause 4 — Negated query**

¬AILearner(Ravi)

- **Step 4: Start resolution**
- We need to find **opposite literals**.
- Clause 1: $Student(Ravi)$
- Clause 3 contains: $\neg Student(x)$
- These can match if: $\boxed{x = Ravi}$
- So Clause 3 becomes:
  $\neg Student(Ravi) \vee \neg Studies(Ravi, AI) \vee AILearner(Ravi)$
- Resolve with: $Student(Ravi)$
- The opposite pair: $Student(Ravi)$ and $\neg Student(Ravi)$ cancel.
- We get:

  ¬Studies(Ravi,AI) VAILearner(Ravi)

- **Step 5: Resolve again**
- We have: $Studies(Ravi, AI)$
- and:

$$\neg Studies(Ravi, AI) \vee AILearner(Ravi)$$

- Cancel the opposite pair.
- We get: *AILearner(Ravi)*

- **Step 6: Resolve with the negated query**
- We have: $AILearner(Ravi)$ and: $\neg AILearner(Ravi)$
- They contradict each other.
- Therefore: $\boxed{\square}$
- The empty clause means **contradiction**.
- Therefore the original query is true: *AILearner(Ravi)*